

WE ARE THE DISTRO

BUILDING A FROM-SCRATCH GLIBC LINUX, PHASE BY PHASE

AUTHOR

Javier Fernández

CONTACT

jfernand@me.com

ISSUED

2026-09-14

ABSTRACT

distro is a Rust CLI that builds a bootable, glibc-based Linux distribution entirely from upstream source, using nothing but the host's own compiler toolchain — no foreign package manager, no bootstrap image, no binary blobs beyond what the kernel itself requires. This report documents what has actually been built and verified in QEMU as of Phase 2: a real kernel, a statically-linked POSIX userland with genuine login/getty authentication, and the seat-management, D-Bus, and device-management daemons a Wayland desktop needs before a compositor can exist at all. It also records the two architectural decisions forced by that work — the pivot from static to dynamic linking, and the staged sysroot that makes chained source builds possible — and the deliberate scope cuts (no libxcb, no legacy multitouch, no tablet support) made to keep each phase's milestone reachable without getting lost in completionism.

DOCUMENT CONTROL

WORKSPACE	Cargo workspace: builder-core (lib) · distroless (musl/BusyBox) · distro (glibc/from-scratch) · distro-init (PID 1)
TARGET	Native x86_64-unknown-linux-gnu — host toolchain, no cross-compilation
VERIFICATION	QEMU/OVMF boot, scripted serial-console interaction, every milestone below reproduced live
STATUS	Phase 2 complete · Phase 3 (graphics) not started

01 WHY THIS EXISTS

Most of the ways to get a custom Linux system booting quickly involve starting from someone else's finished distribution — `debootstrap`, a container base image, an Arch `pacstrap` — and layering changes on top. That approach was explicitly rejected for this project. Every binary that ends up in the built image is compiled here, from source, against nothing but the host machine's own `glibc` and `gcc`. The one deliberate exception is the same one every real distribution makes: the host's **compiler toolchain** — `gcc`, `binutils`, `meson`, `autoconf` — is infrastructure, not distro content, in the same way a bootstrap compiler is infrastructure for a self-hosting language.

READING THIS REPORT

This is a snapshot, not a specification. It describes what has been built and verified, phase by phase, up through Phase 2. §7 lists the phases that come after it (COSMIC, graphics, a Rust toolchain on-target) exactly as scoped in the project's planning document, unstarted.

- § 2 The workspace: four crates, what each one is for.
- § 3 Phase 0 — scaffolding the distro crate.
- § 4 Phase 1 — a minimal `glibc` userland with real login.
- § 5 Phase 2 — seat/session plumbing and the Wayland-core libraries.
- § 6 The two architectural pivots Phase 2 forced: static → dynamic linking, and the staged sysroot.
- § 7 What's still ahead: Phases 3 through 6.
- § 8 How every milestone in this report was actually verified.

02 THE WORKSPACE

CRATE	ROLE
<code>builder-core</code>	Shared library: kernel build (<code>FEATURE_PACKS</code>), disk-image assembly, QEMU test harness, USB writer. Used by both distros unchanged.
<code>distroless</code>	The original, minimal distro: <code>musl</code> + <code>BusyBox</code> + <code>utils</code> , cross-compiled. Not covered in this report.
<code>distro</code>	The subject of this report: a from-scratch <code>glibc</code> distro, native-compiled, aimed eventually at a COSMIC desktop.
<code>distro-init</code>	A from-scratch PID 1 written for this project in ~100 lines of Rust — mounts the virtual filesystems, supervises every daemon below, reaps orphans.

`distro`'s own config (`distro.toml`) does not reuse `distroless`'s top-level `Config` struct — it composes the pieces that are genuinely identical (`KernelConfig`, `ImageConfig`, `UtilsConfig`, all from `builder-core`) with sections of its own (`bash`, `util_linux`, `shadow`, `seatd`, `dbus`, `eudev`, and the six Phase 2 library sections in §5).

DISTRO'S CLI SURFACE (`DISTRO/SRC/CLI.RS`)

```

distro fetch                # download + extract every source tarball
distro build-toolchain      # apt-get the host build tools (once)
distro build-kernel         # builder-core, unchanged
distro build-userland       # everything in §4 and §5
distro assemble-rootfs     # merge it all into build-distro/rootfs
distro make-image           # partition + GRUB + write the disk image
distro test-qemu [--window] # boot it
distro write-usb --device <dev> # dd to real hardware (destructive)
distro all                  # the whole pipeline, in order
```

03

PHASE 0 — SCAFFOLDING

Phase 0 did no source-building at all — its only goal was a working `cargo run -p distro -- build-kernel` end to end, by wiring `distro`’s CLI straight through to `builder-core`’s already-generic kernel/image/QEMU/USB stages. `build-userland` and `assemble-rootfs` were stubs that `bail!`ed with “not yet implemented.” This is the milestone every later phase’s `--force` rebuild still passes through on its way to a fresh image.

04

PHASE 1 — A MINIMAL GLIBC BASE, WITH REAL LOGIN

Phase 1’s target was deliberately narrow: reach a `login:` prompt, authenticate for real, and land in a working `bash` shell — the glibc equivalent of what `distroless` already does with BusyBox, but with no single BusyBox-shaped binary to lean on. Five separate upstream projects fill that one role.

PACKAGE	VERSION	BUILD	ROLE
utils/coreutils	git main	cargo, static	<code>ls / cat / cp /...</code> — the actual applets are a curated feature-flag subset, not the full set (see §8).
bash	5.2.37	autotools, static	Login shell (<code>/bin/sh</code> → <code>bash</code>).
util-linux	2.41.2	autotools, static	<code>agetty</code> , <code>mount</code> , <code>umount</code> only — <code>--disable-all-programs</code> plus explicit <code>--enable-*</code> for just these three.
shadow-utils	4.17.4	autotools, static	<code>login</code> , <code>passwd</code> — real <code>/etc/passwd</code> + <code>/etc/shadow</code> authentication, not BusyBox’s empty-password shortcut.
distro-init	this repo	cargo, static	PID 1: mounts <code>/proc</code> , <code>/sys</code> , <code>/dev</code> ; forks/execs <code>agetty</code> on <code>tty1</code> and <code>ttyS0</code> ; respawns either on exit.

THE AT_EXECFN PATCH
`utils`’ multi-call dispatch (which applet `ls` vs `cat` resolves to) reads the kernel’s `AT_EXECFN` auxval on non-musl Linux instead of trusting `argv[0]`, as a hardening measure. On this build host — and inside this project’s own kernel — `AT_EXECFN` comes back **empty**, so every applet invocation hit “<unknown binary name>” before dispatch even started. `distro/src/stages/fetch.rs` patches a fallback to `argv0` into `src/common/validation.rs` on every fetch, idempotently, with a `bail!` guard if upstream ever changes the code being patched out from under it.

Every Phase 1 binary is **statically linked** (`--disable-shared --enable-static` at configure time, `LDFLAGS=-all-static` at `make` time — plain `-static` breaks `configure`’s own compiler sanity check under `libtool`). One binary in, one binary copied to the rootfs, no shared-library bookkeeping. `/etc/shadow` carries a single `root` account with an empty password field, which makes `login` skip the password prompt entirely rather than reimplementing BusyBox’s separate no-password trick.

VERIFIED
QEMU boot → `distro-init` starts → `agetty` on `tty1` → `login: root` → no password prompt (empty shadow field) → `-bash-5.2#` → `ls /bin | wc -l` → `40`. Reached via `agetty` → `login` → `bash`, not a direct shell spawn.

05 PHASE 2 — SEAT/SESSION PLUMBING, WAYLAND CORE

Phase 2's charter, per the project's own roadmap, was narrow on purpose: get `seatd` and `dbus` — the plumbing a desktop session needs **before** any compositor exists — running on-target, with no compositor yet. What actually shipped is somewhat larger than that stated minimum: alongside the `seatd/dbus` milestone, this phase also built every Wayland-core library Phase 3's compositor will link against, plus `eudev` (device management) and `libinput` (turns raw `evdev` events into pointer/keyboard/touch events), neither of which were in the original two-package milestone but which were pulled forward because `libinput` hard-depends on `libudev` and there is no clean way to build it, or verify anything about it, without that dependency satisfied first.

5.1 THE DAEMONS

PACKAGE	VERSION	BUILD	ROLE
<code>seatd</code>	0.9.3	meson, dynamic	Seat management — mediates access to <code>/dev/input</code> , <code>/dev/dri</code> without requiring root. “Depends only on <code>libc</code> ” per its own README; built dynamically anyway, for consistency with everything after it.
<code>dbus</code>	1.16.2	meson, dynamic	System message bus. Runs as <code>root (-Ddbus_user=root)</code> — no <code>messagebus</code> user exists in this rootfs yet.
<code>eudev</code>	3.2.14	autotools, dynamic	systemd-independent <code>udev</code> fork (the Alpine/Void/Gentoo answer to “I don't have systemd but I need <code>libudev</code> ”). <code>blkid</code> /SELinux/ <code>kmod</code> support all disabled.

All three are supervised by `distro-init` alongside `agetty`: forked, `execed`, and respawned on exit, the same pattern Phase 1 established. `dbus-daemon` runs with `--nofork` and `udev` with `no -d` — both daemonize themselves by default, which would make them invisible to `distro-init`'s own `fork / waitpid` supervision loop. `distro-init` also mounts a `tmpfs` at `/run` (transient state, not part of the persisted rootfs) and runs `udevadm trigger` once at boot as a one-shot **coldplug**: `devtmpfs` already populated `/dev` before `udev` started, but without telling it, so without the trigger `udev`'s own device database would stay empty for anything already present at boot.

VERIFIED

`dbus-send --system ... ListNames` returns a real method-return (`org.freedesktop.DBus, :1.0`) rather than a connection error. `udevadm info --query=all --name=/dev/tty1` after the coldplug trigger returns a fully populated device entry (`DEVPATH / MAJOR / MINOR / SUBSYSTEM`) — proof the trigger worked, not just that `udev` started without crashing.

5.2 THE LINK-TIME LIBRARIES

Nothing below runs as a service — no `distro-init` changes accompany this table. Each is built, installed, and left waiting for Phase 3's compositor to actually link against it; verification at this stage is “builds and installs cleanly against everything before it,” not “does something observable,” since nothing yet calls into them.

PACKAGE	VERSION	BUILD	ROLE
<code>wayland</code>	1.26.0	meson, dynamic	Core wire-protocol libraries (<code>client/server/cursor/egl</code>) and <code>wayland-scanner</code> , the code generator every later Wayland package invokes at its own build time.
<code>wayland-protocols</code>	1.49	meson, dynamic	The protocol XML definitions (<code>xdg-shell</code> and the rest) — data plus a <code>pkg-config</code> file, no library.

PACKAGE	VERSION	BUILD	ROLE
libxkbcommon	1.12.4	meson, dynamic	Keymap compilation. X11 support disabled (no libxcb — see below). xkb-config-root pinned to /usr/share/X11/xkb, though the xkeyboard-config data package itself isn't built yet — see §8.
pixman	0.46.4	meson, dynamic	Software rasterization — Mesa's fallback path and some compositor-side operations not worth doing on the GPU.
libdisplay-info	0.4.0	meson, dynamic	EDID/DisplayID parsing — how a compositor reads a monitor's own supported-mode list.
libevdev	1.13.7	meson, dynamic	libinput's mandatory dependency for reading/writing raw evdev devices.
libinput	1.31.3	meson, dynamic	Raw evdev events → pointer/keyboard/touch/gesture events. libwacom and mtdev support both disabled — see §8.

06 TWO PIVOTS PHASE 2 FORCED

6.1 STATIC LINKING STOPS HERE

Every Phase 1 binary is statically linked — no shared-library management, one file copied into the rootfs, done. `dbus` broke that pattern immediately: it needs `libexpat` for XML parsing and is not meaningfully staticable. Worse, the **rest** of the roadmap sits entirely on the other side of that line — Mesa loads GPU drivers as runtime plugins, GTK/Wayland backends get `dlopen`'d, and fighting to statically link a graphics stack and a desktop environment would mean fighting every upstream build system's own assumptions, indefinitely, for no real benefit. This was surfaced as an explicit decision rather than made silently, and the answer was to switch: Phase 2 onward links dynamically, with the host's own `glibc` (`libc.so.6`), `libexpat.so.1`, and (as of `libinput`) `libm.so.6` copied into the rootfs's `/lib/x86_64-linux-gnu` alongside `/lib64/ld-linux-x86-64.so.2` — confirmed via `ld-linux-x86-64.so.2 --help` to be default, compiled-in dynamic-linker search paths on this host, needing no `ld.so.conf` / `ldconfig` step to work.

WHY NO LD.SO.CONF/LDCONFIG

Shared library search path:
 (libraries located via `/etc/ld.so.cache`)
`/lib/x86_64-linux-gnu` (system search path)
`/usr/lib/x86_64-linux-gnu` (system search path)
`/lib` (system search path)

Both the host libraries and everything the distro builds itself for `/usr/lib/x86_64-linux-gnu` land in a directory the dynamic linker already searches by default on this (Ubuntu) `glibc` build — no cache file, no config file, no `ldconfig` run against the target rootfs required.

6.2 A STAGED SYSROOT, ONCE PACKAGES DEPEND ON EACH OTHER

Phase 1's five packages were independent of one another at build time — each just needed the host's `gcc`. Phase 2 is not: `wayland-protocols` needs `wayland-scanner` on `PATH` at its **own** configure/build time, and `libinput` needs `udev`'s installed `libudev.pc` to find `libudev` via `pkg-config`. The fix has two parts, and getting the split wrong broke a running daemon before it was found.

Part one — the ordinary case. Every package is configured with `--prefix=/usr` (its normal, final, “as if installed for real” prefix) and installed with `DESTDIR=<absolute path to build-distro/sysroot>` — files physically land under the sysroot, but the package's own idea of its prefix stays `/usr`. `PKG_CONFIG_PATH` points every subsequent package's build at the sysroot's installed `.pc` files, and `PKG_CONFIG_SYSROOT_DIR` — `pkg-config`'s own built-in mechanism, not something this project wrote — rewrites the `-I` / `-L` paths those `.pc` files report from their baked-in `/usr/...` to the sysroot's real, on-disk `<sysroot>/usr/...`,

automatically, for ordinary `Cflags / Libs` fields. This is what lets `libinput`'s build actually find and link against `eudev`'s freshly-built `libudev`.

THE BUG THIS DESIGN REPLACED

The first version of this pipeline used an absolute `sysroot` path as `--prefix` for **every** package, reasoning (correctly, for one specific case — see below) that installed `.pc` files need real, resolvable paths. It booted, but `dbus-daemon` immediately crash-looped: `distro-init` kept logging "dbus-daemon exited, respawning", and the actual error was `Failed to open ".../build-distro/sysroot/usr/share/dbus-1/system.conf"`:

No such file or directory

— a literal build-machine path, because `dbus`'s **own compiled-in** default config search path is derived from whatever `--prefix` it was built with, not from `PKG_CONFIG_SYSROOT_DIR` (that only ever affects what **other** packages' builds see when they query `dbus` via `pkg-config` — it says nothing about what `dbus-daemon` looks up on its own at its own runtime). `eudev` had the identical exposure, undetected only because nothing had yet exercised the affected paths. Caught by the same live QEMU boot check every other milestone in this report went through, not by review.

Part two — the one real exception. `wayland-scanner`'s own path is baked into `wayland`'s `.pc` file as a **custom** variable (`wayland_scanner=${bindir}/wayland-scanner`), and `wayland-protocols`' build reads that variable and directly executes whatever path it names. Custom variables are exactly the case `PKG_CONFIG_SYSROOT_DIR` does **not** rewrite (only `Cflags / Libs`, and only for a variable the package itself templated with `${pc_sysrootdir}` — `wayland`'s own build doesn't do that for this one). With `--prefix=/usr`, the baked path reads the literal string `/usr/bin/wayland-scanner`, which does not exist anywhere on this host — running it is not optional, `wayland-protocols`' own build does so directly. So `wayland` alone is built with a real, absolute, on-disk `--prefix=<sysroot>/usr` and installed there directly, no `DESTDIR`. This is safe specifically for `wayland`, and not a pattern to reach for by default: none of its own **shared libraries** do a prefix-derived runtime lookup the way `dbus` / `eudev`'s daemons do — a `.so`'s `SONAME`-based linking doesn't care what `--prefix` built it, only `wayland-scanner`'s baked tool-path variable does.

DISTRO/SRC/STAGES/USERLAND.RS — THE ORDINARY CASE (SEATD, DBUS, EUDEV, AND EVERYTHING AFTER WAYLAND)

```
fn meson_build_and_install(cfg: &Config, dir: &Path, extra_args: &[&str]) -> Result<()> {
    let destdir = sysroot_abs(cfg)?;
    let mut setup = Command::new("meson");
    setup.arg("setup").arg("build").arg("--prefix=/usr").args(extra_args);
    sysroot_env(cfg, &mut setup)?; // PKG_CONFIG_PATH + PKG_CONFIG_SYSROOT_DIR + PATH
    run_in(dir, &mut setup)?;

    let mut build = Command::new("ninja");
    build.arg("-C").arg("build");
    sysroot_env(cfg, &mut build)?;
    run_in(dir, &mut build)?;

    let mut install = Command::new("ninja");
    install.arg("-C").arg("build").arg("install");
    sysroot_env(cfg, &mut install)?;
    install.env("DESTDIR", destdir); // real /usr baked in; staged elsewhere on disk
    run_in(dir, &mut install)
}
```

07 WHAT'S STILL AHEAD

Unchanged from the project's original roadmap — none of the following has been started:

- PHASE 3 Graphics, scoped first to QEMU's `virtio-gpu`: Mesa, built against the Wayland-core libraries §5 already produced.
- PHASE 4 `rustup` / `cargo` on-target, plus a curated Rust-CLI-tools suite (`ripgrep`, `bat`, `eza`, ...), cross-built on the host and copied in like everything in §4–5.
- PHASE 5 COSMIC itself — `cosmic-comp`, `cosmic-session`, `cosmic-panel`, `cosmic-greeter`, minimal subset first.
- PHASE 6 Expand: more COSMIC components, real GPU drivers beyond `virtio-gpu`, audio, networking UI.

The single biggest open risk flagged in the original roadmap — whether `seatd` alone suffices for a real COSMIC session, or whether `systemd/logind` ends up unavoidable — remains genuinely open. Phase 2 proves `seatd` starts and holds a seat; it does not prove COSMIC will accept it as `logind`’s replacement.

08

VERIFICATION METHODOLOGY

Every milestone claimed in §4–5 was reproduced live, not inferred from a successful compile. The method, unchanged since Phase 1: a Python harness opens a PTY (`pty.openpty()`, with `TIOCSWINSZ` set — QEMU’s serial console renders nothing useful at a 0×0 terminal size), launches `qemu-system-x86_64` with the built image attached over `virtio` and OVMF for UEFI boot, and drives the console with a small state machine keyed on regexes over the accumulated output (`login:` , a shell prompt, a sentinel string echoed after each command) rather than fixed sleeps.

A REAL BUG THIS CAUGHT

`assemble-rootfs` initially failed with “No such file or directory” trying to copy the freshly built `distro-init` binary. Cause: this build host sets `CARGO_TARGET_DIR` to a shared cache outside the repo (`/mnt/rust-cache/target`), which `cargo` honors — so the binary was never at the hardcoded `target/...` path the `rootfs-assembly` code assumed. Fixed by reading `CARGO_TARGET_DIR` at the same call site `cargo` itself does. Caught by an actual failing `assemble-rootfs` run, not by review.

KNOWN, DELIBERATE GAPS

- `utils`’ built feature set does not include `grep` or `sed` — despite both names appearing in `rootfs.rs`’s `COREUTILS_APPLETS` symlink list. They aren’t part of `coreutils`’ scope upstream in the first place; the symlinks exist but dangle. Not yet fixed.
- `libxcb` was not built. Its only real consumer on a Wayland-native target is XWayland compatibility, not currently planned; three more from-source packages (`libxcb` itself, `libXau`, `libXdmcp`) were not worth building for a checklist item with no near-term user.
- `mtdev` (legacy multitouch “protocol A” translation) and `libwacom` (tablet identification) were both skipped when building `libinput` — niche hardware support, easy to add back later if a real device needs it.
- `libxkbcommon`’s `xkb-config-root` points at the FHS-standard `/usr/share/X11/xkb`, but the `xkeyboard-config` data package that would actually populate that path has not been built — real keymap compilation will not work until it is, which is not needed until Phase 3 has a compositor to test it with.